SIMATS ENGINEERING
ASSESSMENT TOOL 3

NAME: R Mohnish **REG NO: 192411112**
COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA4304**

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Responsive Web Design (RWD) is a web development approach that allows a website to automatically adapt its layout and content to different screen sizes such as mobile phones, tablets, and desktops. It ensures a consistent user experience across all devices.

Importance

- Improves user experience on all devices.
- Increases website accessibility and usability.
- Reduces maintenance cost by using a single website.

- Enhances SEO and mobile-friendliness.

Key CSS Techniques

1. **Fluid Layouts** – Use percentages instead of fixed widths.
2. **Flexible Images** – Images resize within containers.
3. **Media Queries** – Apply styles based on screen size.
4. **Flexbox and Grid** – Create responsive layouts easily.

Using Flexbox/Grid

- **Flexbox** is used for one-dimensional layouts (row or column).
- **Grid** is used for two-dimensional layouts (rows and columns).

Example Layout

```
<div class="container">
  <div class="item">Header</div>
  <div class="item">Content</div>
  <div class="item">Sidebar</div>
</div>
.container{
    display:flex;
    flex-wrap:wrap;
}
.item{
    flex:1 1 300px;
    padding:10px;
}
```

Improvements

- Use touch-friendly buttons on mobile devices.
- Optimize images for faster loading.
- Use responsive navigation menus.
- Ensure readable font sizes across devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Client-side validation is the process of checking user input in the browser before sending data to the server.

Purpose

- Prevents invalid data submission.
- Provides instant feedback to users.
- Reduces server workload.
- Improves user experience.

JavaScript Event Handling in Validation

JavaScript uses events such as:

- onsubmit
- oninput
- onblur

These events trigger validation functions when users interact with form fields.

Role of Regular Expressions

Regular Expressions (Regex) define patterns for validating inputs.

Email Regex:

/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/

Phone Number Regex:

/^[0-9]{10}$/

Example

```
<form onsubmit="return validate()">
  <input type="email" id="email">
  <input type="submit">
</form>


<script>
function validate(){
    let email=document.getElementById("email").value;
    let pattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;


    if(!pattern.test(email)){
        alert("Invalid Email");
        return false;
    }
    return true;
}
```

</script>

Improvements

- Display error messages beside fields.
- Use HTML5 validation attributes.
- Validate data again on the server side.
- Provide real-time validation feedback.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

The **CSS Box Model** describes how elements are displayed on a webpage.

Components of Box Model

1. **Content** – Actual text or image.
2. **Padding** – Space around content.
3. **Border** – Surrounds padding.
4. **Margin** – Space outside the border.

CSS Positioning Techniques

- **Static** – Default positioning.
- **Relative** – Positioned relative to normal location.
- **Absolute** – Positioned relative to nearest ancestor.
- **Fixed** – Fixed to viewport.
- **Sticky** – Sticks while scrolling.

Causes of Layout Breaking

- Fixed widths larger than screen size.
- Improper positioning.
- Large images without responsiveness.
- Missing media queries.
- Overflowing content.

Using Media Queries

```
@media screen and (max-width:768px){
    .container{
        flex-direction:column;
    }
}
```

Best Practices

- Use Flexbox or Grid layouts.
- Avoid fixed widths.
- Use relative units (% , rem , em).
- Make images responsive.
- Test on different screen sizes.

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

The **Document Object Model (DOM)** is a tree-like representation of an HTML document. It allows JavaScript to access and modify webpage elements dynamically.

Role of DOM in Web Development

- Access webpage elements.
- Change content and styles.
- Add or remove elements.
- Respond to user interactions.

DOM Manipulation Using JavaScript

JavaScript can update content without reloading the page.

Example

```
<p id="msg">Welcome</p>

<button onclick="changeText()">Click Me</button>



<script>

function changeText(){

   document.getElementById("msg").innerHTML =

   "Content Updated Successfully!";

}

</script>
```

How It Works

- JavaScript selects an element using getElementById().
- The content is modified using innerHTML.
- Changes appear instantly without page refresh.

Improvements

- Use AJAX/Fetch API for loading data asynchronously.
- Minimize unnecessary DOM updates.
- Use event delegation for better performance.
- Show loading indicators during updates.
- Optimize scripts for faster user interaction.

Result: Dynamic DOM manipulation provides a faster, more interactive, and user-friendly website experience.

Code of Conduct:

I **M.NAVEEN /192411134** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: naveen